

Guía de Resolución: Incidentes Simulados y Mitigación con Laboratorio Real

Objetivo General

Aplicar conocimientos de seguridad ofensiva y defensiva para detectar, clasificar, mitigar y prevenir múltiples incidentes en un entorno real simulado (laboratorio con WAF, backend, frontend, CSP, etc.).

Requisitos Previos del Lab

- Acceso al entorno Docker con los siguientes servicios corriendo:
 - **frontend** (puerto 8080)
 - **backend** (puerto 8000)
 - **waf** (puerto 8001)
 - **csp** (puerto 8003)
 - **sast** (puerto 8002)
 - **attacker** (automatiza ataques desde red interna)

Paso a Paso: Reproducción y Análisis de Incidentes

1. Fuerza bruta / SQL Injection en Login

Simulación

```
curl -X POST http://localhost:8001/login \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"username":"admin", "password":"pass"}'
```

```
curl -X POST http://localhost:8001/login \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"username":"admin\" OR \"1\"=\"1", "password":"pass"}'
```

Mitigación

- El **WAF detecta patrones de SQLi** y bloquea la IP si excede el límite (`BLOCK_LIMIT=100`).
- Analiza los logs del WAF: `/logs/waf.log` o salida de consola.

2. XSS Persistente en Comentarios

Simulación

```
curl -X POST http://localhost:8001/comment \  
-H "Content-Type: application/json" \  
-d '{"comment":"<script>alert(\"xss\")</script>"}'
```

Mitigación

- El **WAF bloquea contenido con etiquetas** `<script>`, `javascript:`, `onerror`, etc.
- Verifica con:

```
curl http://localhost:8001/comments
```

3. Acceso a Endpoint sin Autenticación

Simulación

```
curl http://localhost:8001/admin/export
```

Mitigación

- Backend debe implementar control de acceso basado en roles (RBAC).
- Asegúrate de que el backend no devuelva información sin token JWT o rol adecuado.

4. Análisis SAST (código inseguro)

Simulación

`curl http://localhost:8002/scan`

Mitigación

- El microservicio `sast` devuelve vulnerabilidades estáticas simuladas (hardcoded passwords, falta de validación, etc).
- Usa los resultados para justificar refactorización.

5. Uso de librerías vulnerables

- Simulado en el frontend (por ejemplo, incluye `jquery-1.4.2.min.js`).
- Verifica en navegador o con:

`curl http://localhost:8080`

Mitigación

- Identifica dependencias obsoletas.
- Usa `npm audit`, `retire.js`, o `snyk`.

Actividades del Informe

Parte 1 – Clasificación de los incidentes

Incidente	Tipo de Acción
Intentos de login repetidos	Mitigación + Prevención
XSS persistente	Mitigación + Prevención
Acceso sin rol al export	Mitigación inmediata
Librería vulnerable no parcheada	Prevención
Mensajes de error con estructura de base de datos	Prevención

Parte 2 – Propuesta Técnica por Incidente

Incidente	Mitigación	Prevención
Fuerza Bruta / SQLi	WAF con regex y bloqueo por IP	Límite de intentos + CAPTCHA
XSS Persistente	Filtro WAF con regex	Escapado y sanitización del lado del servidor
Acceso sin autorización	Endpoint protegido con JWT y rol	Política de acceso en backend y pruebas automatizadas

Librería vulnerable Auditoría con `retire.js` o `npm audit` CI/CD con escaneo de dependencias

Mensajes de error Manejo genérico de errores en backend Desactivar `debug=True` en producción

Parte 3 – Prioridad de Intervención

Prioridad	Incidente	Justificación
1 Alta	XSS persistente	Impacto directo sobre usuarios (robo de sesión, phishing)
2 Alta	Fuerza bruta / SQLi	Riesgo de acceso no autorizado o corrupción de datos
3 Media	Acceso sin control a <code>/admin/export</code>	Puede exponer datos sensibles sin autenticación
4 Media	Mensajes con errores técnicos	Puede facilitar ataques dirigidos si revela estructura interna
5 Baja	Librerías vulnerables	Exposición a largo plazo si no se explota directamente